

Learning When to Act: Speculative Tool Execution from the Hidden States of Reasoning Models

Anonymous ACL submission

Abstract

Reasoning models pay for every tool call twice: wall-clock time while the tool runs, and thinking tokens spent deriving a decision whose outcome is often predictable. We present [METHOD-NAME], a speculator that reads the frozen main model’s hidden states and predicts upcoming tool calls, their arguments, and the benefit of acting early. [PLACEHOLDER: headline numbers.]

1 Introduction

Reasoning models use external tools by interleaving long chains of thought with tool calls. The call itself is synchronous: the model thinks for hundreds of tokens, commits to a call, and then waits for the result before thinking further. This design bills every call twice. The user pays wall-clock time while the tool runs, and pays again for the thinking tokens the model spends deriving a decision whose outcome is often predictable, especially when an agent serves many similar requests over its deployment lifetime.

Speculative execution is the natural answer, and recent systems have started to apply it to tool calls. SPORK (TODO: [verify authors, 2026h](#)) forks the model to act as its own predictor of the upcoming call; PASTE (TODO: [verify authors, 2026d](#)) mines control-flow patterns from past workflows and pre-executes the calls they imply; SIA (TODO: [verify authors, 2026g](#)) speculatively executes calls whose arguments may still be incomplete. These systems differ in what they predict, but they share one design choice: the decision of *when* to act is driven by output-side confidence or by workflow structure, never by the state of the model that is doing the thinking. Our measurements show this choice is not a detail. Injecting a tool result 25–50 tokens before the model would issue the call yields *negative* returns: the model has entered the decision-critical span of its reasoning, and the interruption costs

more than the head start saves. We call this span the dead zone. Confidence-triggered firing lands in it systematically, because prediction confidence peaks exactly where the model is about to commit. The benefit of speculation is concentrated where confidence is lowest: very early in the thinking, far ahead of the call point.

This paper argues that the missing signal lives in the model’s hidden states, and that it can be learned. We present [METHOD-NAME], a speculator: a set of lightweight heads that read the frozen main model’s hidden states at every decoding step and predict, jointly, whether a call is coming, which tool it will use, what its arguments will be, and how much is gained by acting now. The argument head follows the shape of the argument space: calls with no arguments are speculated most aggressively; calls whose arguments come from a closed candidate set visible in the context are checked against that set before speculation; free-form arguments are speculated conservatively or not at all. The timing head is trained on labels produced by oracle replay: we re-run stored trajectories, inject the known-correct result at varying lead times, and record the realized benefit, so the head learns the dead-zone geometry directly instead of inheriting it from confidence.

The speculator deploys in two modes with separate accounting. In prefetch mode, a triggered call runs in the background while thinking continues unchanged; the token stream is untouched, accuracy is preserved by construction, and the tool’s latency is hidden behind decoding. In truncate mode, a triggered call ends the thinking immediately; this saves thinking tokens and trades a controlled fraction of wrong calls, tunable by the firing threshold θ . On [BENCHMARK], truncate mode at $\theta = 0.8$ saves [PLACEHOLDER]% of thinking tokens with [PLACEHOLDER] accuracy change, and prefetch mode hides [PLACEHOLDER]% of tool latency at zero accuracy cost.

Our contributions: (1) a measurement of the injection-timing curve showing a dead zone before the call point, which output-side confidence cannot avoid; (2) [METHOD-NAME], a hidden-state speculator with a learned timing head and a tiered argument head; (3) an evaluation protocol that reports speed and accuracy jointly, as savings curves against accuracy-loss curves rather than prediction accuracy alone; (4) [PLACEHOLDER: benchmark / learning-curve contribution, pending merged-track design].

2 Related Work

Speculative execution of tool calls. SPORK (TODO: verify authors, 2026h) needs no training: it forks the model mid-thought and lets the fork race ahead to predict the full call, reaching [PLACEHOLDER: 74.6–99.6%] tool-name accuracy, at the price of a second decoding stream. PASTE (TODO: verify authors, 2026d) and B-PASTE (Song, 2026) predict from mined workflow regularities and schedule speculation by estimated critical-path reduction; the estimate comes from workflow structure, so it cannot see effects that exist only in the model’s reasoning, the dead zone among them. Cost-aware schedulers (Fareed, 2026) price speculation in dollars with an expected-value rule, again without reading the model. Speculate-with-Memory (TODO: verify authors, 2026f) adds an external memory to its predictor but still treats the main model as a black box. Against this line, our speculator reads the main model’s hidden states, adds no second decoding stream, and learns timing from realized benefit rather than structure or confidence.

Reading internal states for tool decisions. When2Tool (TODO: verify authors, 2026j) shows that whether a tool is needed is linearly decodable from hidden states (AUROC 0.89–0.96), but reads once, before generation, and stops at the binary decision. PPRAG (TODO: verify authors, 2026e) attaches a predictor to middle-layer states and prefetches retrievals mid-decoding, cutting end-to-end latency by 43.5%; retrieval is its only tool, and a query its only argument. A recent probe study (TODO: verify authors, 2026i) decodes the dependency topology between future calls from the residual stream and states explicitly that it makes no behavioral-control claims. OLIVIA (TODO: verify authors, 2026c) runs a contextual bandit over frozen hidden states to pick among candidate ac-

tions in repeated deployments, without speculation or prefetching. These works establish that the signals we need exist; none closes the loop from signal to speculative execution of general tool calls with arguments, timing, and post-hoc verification.

Asynchronous execution and mid-generation injection. AsyncLM (Gim et al., 2024) removes the wait: calls run asynchronously and an interrupt protocol notifies the model in-flight, cutting task latency by up to $5.4\times$. The call is still authored by the model itself; AsyncLM removes waiting, not thinking. SIA (TODO: verify authors, 2026g) injects finished results into the context following the same mechanism. IdleSpec (TODO: verify authors, 2026b) fills tool-wait idle time with candidate plans, trading idle compute for accuracy where we trade prediction for latency. Ghost-call privacy analysis (TODO: Mohammadi et al., 2026) shows that speculative calls leak at issue time, which motivates the read-only restriction in our prefetch mode. Our speculator composes with this line: a speculated call can be dispatched on an asynchronous substrate, and our verification step reconciles it with the call the model eventually issues.

Agents on repeated tasks. AgentCL (TODO: verify authors, 2026a) motivates wasted inference on recurring tasks without measuring it; caching layers reuse results but not decisions; Speculate-with-Memory (TODO: verify authors, 2026f) draws a learning curve in estimated latency rather than wall-clock time.

3 Method

4 Evaluation Protocol: Joint Speed–Accuracy Learning Curves

Existing streaming benchmarks measure whether agents get more accurate with experience; the cost of that experience is reported incidentally, if at all. Our protocol makes amortized cost a first-class axis: every reported accuracy number is paired with the cumulative tokens spent reaching it, on task streams whose similarity structure is controlled. The protocol answers three questions: how much does memory save as tasks repeat, what does competence cost, and when does remembered experience actively hurt.

4.1 Task Streams and the Similarity Ladder

A benchmark instance is a stream of K episodes. Every subject runs each stream twice under iden-

tical seeds and task instances: a MEM arm with its memory mechanism enabled, and a NOMEM arm with memory disabled. All comparisons are paired at the (run, position, task instance) level; the NOMEM arm absorbs the difficulty fluctuations of the stream so that differences read out the memory mechanism alone.

Streams are drawn at five similarity levels, numbered by the degree to which memory *should* help. L0 streams contain unrelated tasks (pairwise similarity < 0.3): a well-behaved memory should be inert here, and any accuracy drop on L0 measures the mechanism’s indiscriminate overhead. L1 streams contain partially similar tasks (same task type, different objects or entities). L2 streams contain repeats, from same-parameters-different-scene up to exact repetition of one instance. L3 and L4 are structured probes rather than points on the monotone axis. An L3 task is composed entirely of components that each appeared in the stream’s history while the combination itself is new; it separates memories that store trajectories from memories that store reusable skills. An L4 task is a trap: superficially as similar to a seen task as a genuine repeat, but constructed so that the stored solution fails. Admission to L4 requires both conditions to hold by construction – embedding similarity to the seed task at or above the median similarity of genuine L2 repeat pairs, and a mechanical replay of the seed’s solution that verifiably fails on the new task. Each admitted trap records its divergence depth $d^* \in [0, 1]$, the fraction of the replayed trajectory executed before the first contradiction signal, and is tiered accordingly: T1 ($d^* < 0.25$, fails on contact), T2 (fails midway), and T3 (replay completes without any contradiction signal and only the final grade fails). T3 is the dangerous tier: the failure is silent in every process metric.

The ladder is not a synthetic worry. In a 357k-turn corpus of real coding agent sessions, 68.4% of adjacent-session pairs have tool-composition similarity above 0.8, another 17.2% fall between 0.3 and 0.8, and 96.1% of sessions have a near-duplicate predecessor within a 50-session window – repeated similar work is the dominant regime of real agent workloads, not an assumption. Among high-similarity pairs, 10.8% cross project boundaries: naturally occurring look-alikes with different ground truth, the L4 phenomenon in the wild.

4.2 Metrics

Let $u_{r,k} \in [0, 1]$ and $c_{r,k} > 0$ be the grade and cost of episode k in run r ; cost is input plus output tokens, the only currency comparable across serving hardware. Wall-clock is reported within one hardware configuration only. P_k denotes the paired grid at position k : runs whose two arms both completed the full stream and both graded ok at k .

Speedup@k. The savings curve is the ratio of sums over the paired grid,

$$\text{Sp@k} = 1 - \frac{\sum_{r \in P_k} c_{r,k}^{\text{MEM}}}{\sum_{r \in P_k} c_{r,k}^{\text{NOMEM}}}, \quad (1)$$

not the mean of per-run ratios, which is unstable under the heavy-tailed per-episode savings we observe in practice. Sp@k is reported only together with $\text{Acc@k} = \frac{1}{|P_k|} \sum_{r \in P_k} u_{r,k}^{\text{MEM}}$ on the same grid: an agent that fails fast scores spectacular savings, so a savings number without its paired accuracy is meaningless. Positions with $|P_k|$ below a preregistered floor are left as gaps, never filled by surviving runs.

Cost-to-competence. Competence at position k is the trailing window mean $\hat{p}_{r,k} = \frac{1}{w} \sum_{j=k-w+1}^k u_{r,j}$ ($w=3$); the attainment position is $k_r^*(\theta) = \min\{k \geq w : \hat{p}_{r,k} \geq \theta\}$. The tuition to competence θ is

$$\text{CTC}_r(\theta) = \sum_{j=1}^{\min(k_r^*, K)} c_{r,j}, \quad (2)$$

a sum that deliberately includes failed episodes: tuition includes the retakes. Runs that never attain θ are censored at the full-stream budget and reported with a $\geq$ sign, alongside the attainment rate $\rho(\theta)$; averaging over attaining runs only is the textbook form of survivorship bias and is excluded by construction.

Negative-transfer index. On the L4 probe segment (seeding episodes excluded), with both arms paired and ok-graded,

$$\text{NTI}_{\text{acc}} = \bar{u}^{\text{NOMEM}} - \bar{u}^{\text{MEM}}, \quad (3)$$

$$\text{NTI}_{\text{cost}} = \frac{\bar{c}^{\text{MEM}} - \bar{c}^{\text{NOMEM}}}{\bar{c}^{\text{NOMEM}}}, \quad (4)$$

both signed so that positive means memory hurt. The two components are never merged: faster-but-wrong (T3 signature) and slower-and-wrong (T2 signature) are different failure modes that a weighted sum would average away. Results are

stratified by trap tier, yielding a dose–response curve, and accompanied by the misuse rate: the fraction of probe episodes whose action prefix copies the stored solution past its divergence depth. A significant NTI_{acc} with near-zero misuse rate indicates the drop has another cause and may not be attributed to experience misuse.

4.3 Aggregation Rules

Five rules, implemented as assertions in the released analysis code rather than as conventions. (1) Per-position statistics use only runs that completed all K episodes. (2) Every MEM–NOMEM comparison is paired on run, position, and task instance; a cell missing either arm is dropped whole. (3) Headline metrics are computed on ok-graded episodes, and the ok-rate of every condition is itself reported: a condition that inflates its ok-only mean by shedding hard episodes is exposed by its ok-rate. (4) Censoring is explicit: no reachers-only means, ever. (5) Wall-clock never aggregates across hardware; tokens are the cross-machine currency.

4.4 Baselines

Every subject is bracketed by three memoryless references. The NOMEM floor is the paired arm defined above. The oracle-replay ceiling replays the cheapest known successful solution of an instance at zero token cost once one exists in the stream’s history; no memory system can beat free replay. The ceiling is informative precisely because it is narrow: verbatim replay is defined only where instances truly repeat, and on our L2 streams it saves at most 21% of stream tokens while leaving success unchanged – the expensive episodes are the failures, which replay cannot fix. Subjects must therefore be read against the ceiling of their level: savings beyond the ceiling can only come from generalization, not recall. The third reference stuffs the full interaction history into the context window, a baseline prior work found surprisingly strong; it prices the alternative of not having a memory mechanism at all, only a long context.

5 Experiments

6 Conclusion

Limitations

TODO.

References

- Faisal Fareed. 2026. [Cost-aware speculative execution: TODO verify exact title](#). *Preprint*, arXiv:2606.07846.
- In Gim, TODO Lee, and TODO Zhong. 2024. [Asynchronous LLM function calling](#). *Preprint*, arXiv:2412.07017.
- Yanfei Song. 2026. [B-PASTE: TODO verify exact title \(bounded-beam speculative scheduling\)](#). *Preprint*, arXiv:2604.16469.
- TODO Mohammadi, TODO Klein, TODO Arora, and TODO Bindschaedler. 2026. [Ghost tool calls: TODO verify exact title](#). *Preprint*, arXiv:2606.02483.
- TODO: verify authors. 2026a. [AgentCL: TODO verify exact title](#). *Preprint*, arXiv:2606.02461.
- TODO: verify authors. 2026b. [IdleSpec: TODO verify exact title](#). *Preprint*, arXiv:2605.22154.
- TODO: verify authors. 2026c. [OLIVIA: TODO verify exact title \(contextual bandit over frozen hidden states\)](#). *Preprint*, arXiv:2605.11169.
- TODO: verify authors. 2026d. [PASTE: TODO verify exact title \(pattern-based speculative tool execution\)](#). *Preprint*, arXiv:2603.18897.
- TODO: verify authors. 2026e. [Predictive prefetching for retrieval-augmented generation: TODO verify exact title](#). *Preprint*, arXiv:2605.17989.
- TODO: verify authors. 2026f. [Speculate with memory: TODO verify exact title](#). *Preprint*, arXiv:2607.12236.
- TODO: verify authors. 2026g. [Speculative interaction agents: TODO verify exact title](#). *Preprint*, arXiv:2605.13360.
- TODO: verify authors. 2026h. [SPORK: TODO verify exact title \(training-free self-speculative forking for tool calls\)](#). *Preprint*, arXiv:2607.03333.
- TODO: verify authors. 2026i. [TODO verify exact title \(linear decodability of tool-call dependency topology\)](#). *Preprint*, arXiv:2605.25310.
- TODO: verify authors. 2026j. [When2Tool: TODO verify exact title](#). *Preprint*, arXiv:2605.09252.

A Appendix